



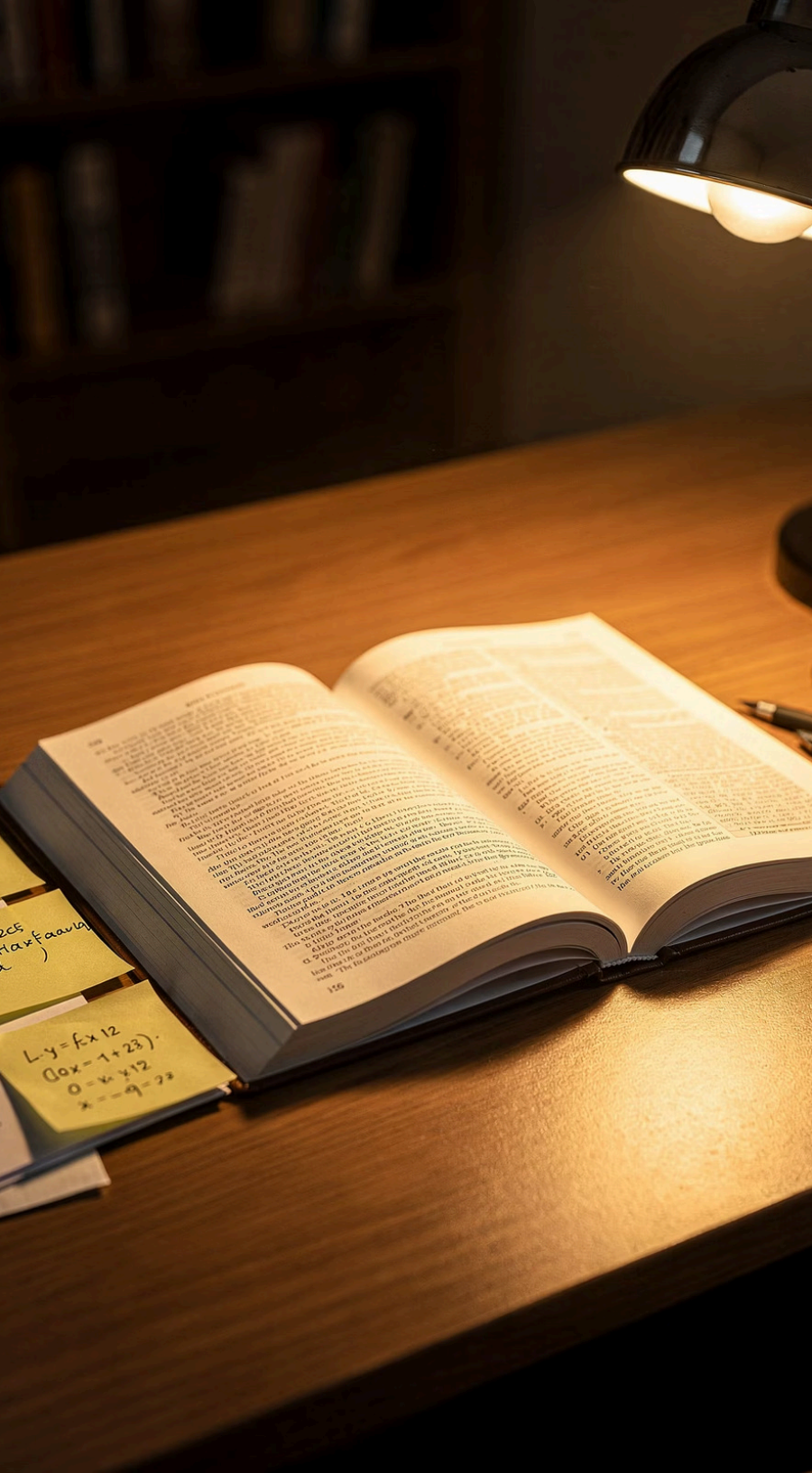
SQL Operators

Unit I · Lecture 5 · MySQL (SQL/PL-SQL)

Presented by **Renuka Parmar** · BCA Program

SQL FUNDAMENTALS

BEGINNER FRIENDLY



Previous Lecture Recap

Database & DBMS

Structured storage and management of data using RDBMS

SQL & MySQL

Structured Query Language; MySQL as the popular RDBMS engine

SQL Commands

DDL, DML, DCL, TCL — command categories and their roles

SQL Data Types

INT, VARCHAR, DATE, FLOAT — defining column data precisely

Today we build on this foundation to explore **SQL Operators** — the tools that make queries powerful.

Learning Objectives

By the end of this lecture, you will be able to:

01

Define SQL Operators

Understand what operators are and why they are essential in SQL queries

02

Classify All Operator Types

Identify Arithmetic, Relational, Logical, and Special operators

03

Apply Operators in Queries

Write correct SQL using operators in SELECT, WHERE, and HAVING clauses

04

Choose the Right Operator

Distinguish between operator types and avoid common beginner mistakes

05

Solve Practical Problems

Match real-world requirements to the correct operator with confidence

What is an Operator?

An **operator** is a symbol or keyword that tells SQL how to process data — compare values, perform calculations, or combine conditions.

☞ Think of operators as the **verbs of SQL** — they perform the action on your data.



Without Operators

You can only fetch *all* rows — no filtering, no logic, no control

With Operators

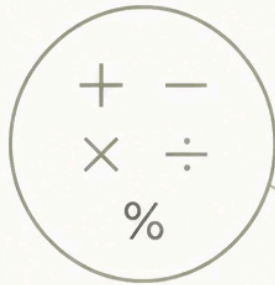
You ask precise questions: "Give me students **where marks > 75**"

Classification of SQL Operators

Each category has a distinct purpose. Together, they form the complete toolkit for writing powerful SQL queries.

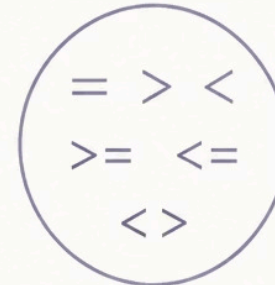
1) Arithmetic Operators

(+, -, *, /, %) for mathematical calculations



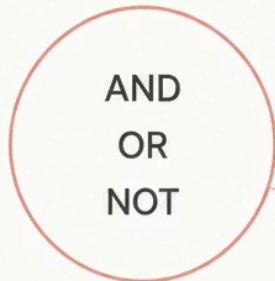
2) Relational Operators

(=, >, <, >=, <=, <>) for comparing values



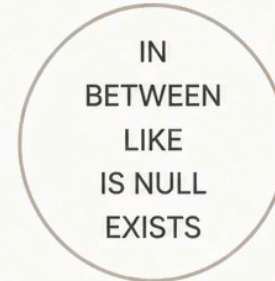
3) Logical Operators

(AND, OR, NOT) for combining conditions



4) Special Operators

(IN, BETWEEN, LIKE, IS NULL, EXISTS) for advanced filtering



Arithmetic Operators

Used to perform **mathematical calculations** on numeric column values directly inside SQL queries.

Operator	Name	Example	Result
+	Addition	<code>SELECT salary + 500</code>	Increased pay
-	Subtraction	<code>SELECT marks - 10</code>	Deducted marks
*	Multiplication	<code>SELECT price * qty</code>	Total amount
/	Division	<code>SELECT salary / 12</code>	Monthly wage
%	Modulus	<code>SELECT roll % 2</code>	Odd/Even check

 **Best Use:** Calculating totals, averages, bonuses, or derived columns on the fly — without altering the original data.

Relational (Comparison) Operators

Used in the **WHERE clause** to compare a column value against a condition. Returns TRUE or FALSE for each row.

= Equal to marks = 80	> Greater than salary > 30000	< Less than age < 20
>= Greater or equal marks >= 50	<> / != Not equal dept <> 'HR'	

Real-World Use: `SELECT * FROM students WHERE marks >= 60;` — fetches only passing students.

Logical Operators

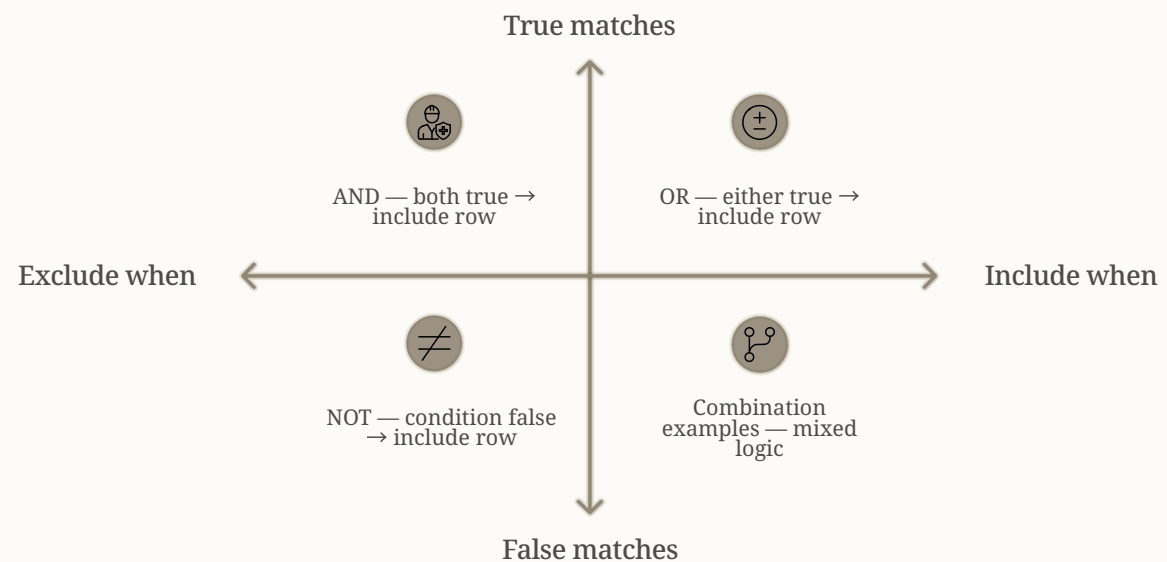
Logical operators **combine multiple conditions** in a WHERE clause, giving you precise control over which rows are returned.

O p e r a t o r	Meaning	Return s TRUE when...
A N D	Both conditions must be true	All conditions match
O R	At least one condition is true	Any one matches
N O T	Reverses the condition	Condition is false

Example:

```
SELECT * FROM emp WHERE  
dept='IT' AND salary > 40000;
```

```
SELECT * FROM emp WHERE  
city='Delhi' OR city='Pune';
```



Special Operators

These operators handle **advanced filtering scenarios** that arithmetic or relational operators cannot cover alone.



IN

Matches any value in a list.

WHERE city IN ('Delhi','Mumbai')



BETWEEN

Checks a range (inclusive).

WHERE marks BETWEEN 50 AND
75



LIKE

Pattern matching with % and _.

WHERE name LIKE 'A%'



IS NULL

Checks for missing values.

WHERE email IS NULL

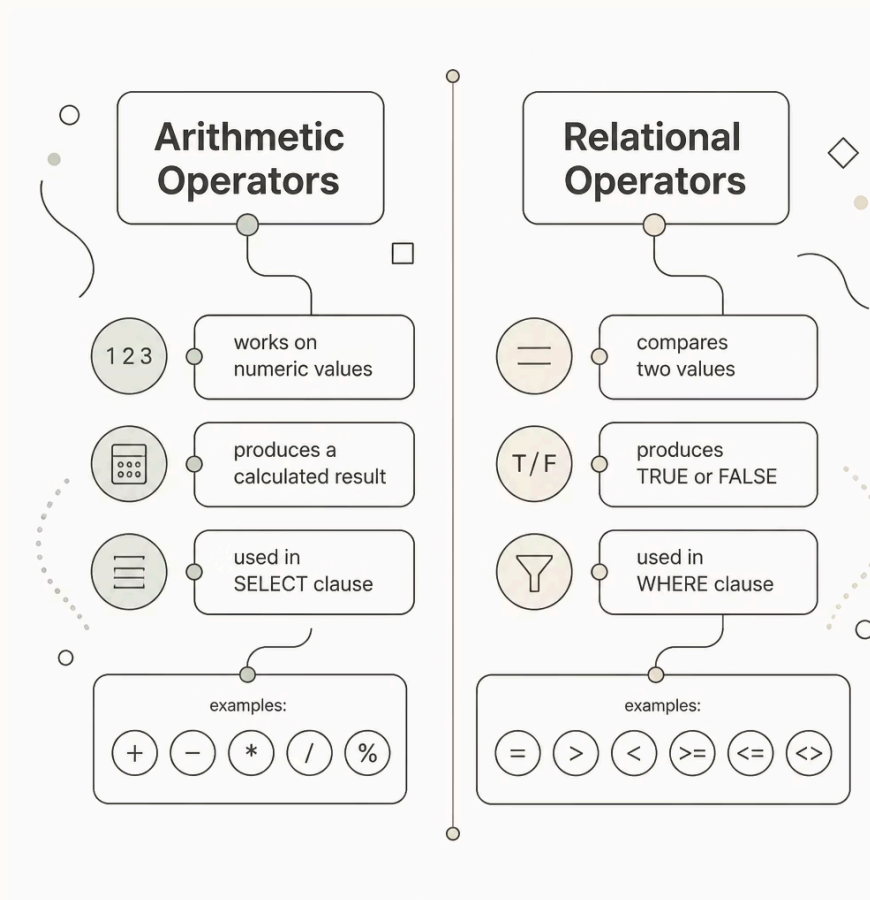


EXISTS

Checks if a subquery returns
rows.

WHERE EXISTS (SELECT ...)

Arithmetic vs Relational Operators



Key Distinction

Arithmetic operators transform data — they produce a new numeric value as output.

Relational operators evaluate data — they produce a boolean (TRUE/FALSE) that filters rows.

Arithmetic Example

```
SELECT salary * 1.10 AS new_salary
```

→ Calculates a 10% raise

Relational Example

```
WHERE salary > 50000
```

→ Filters high earners only

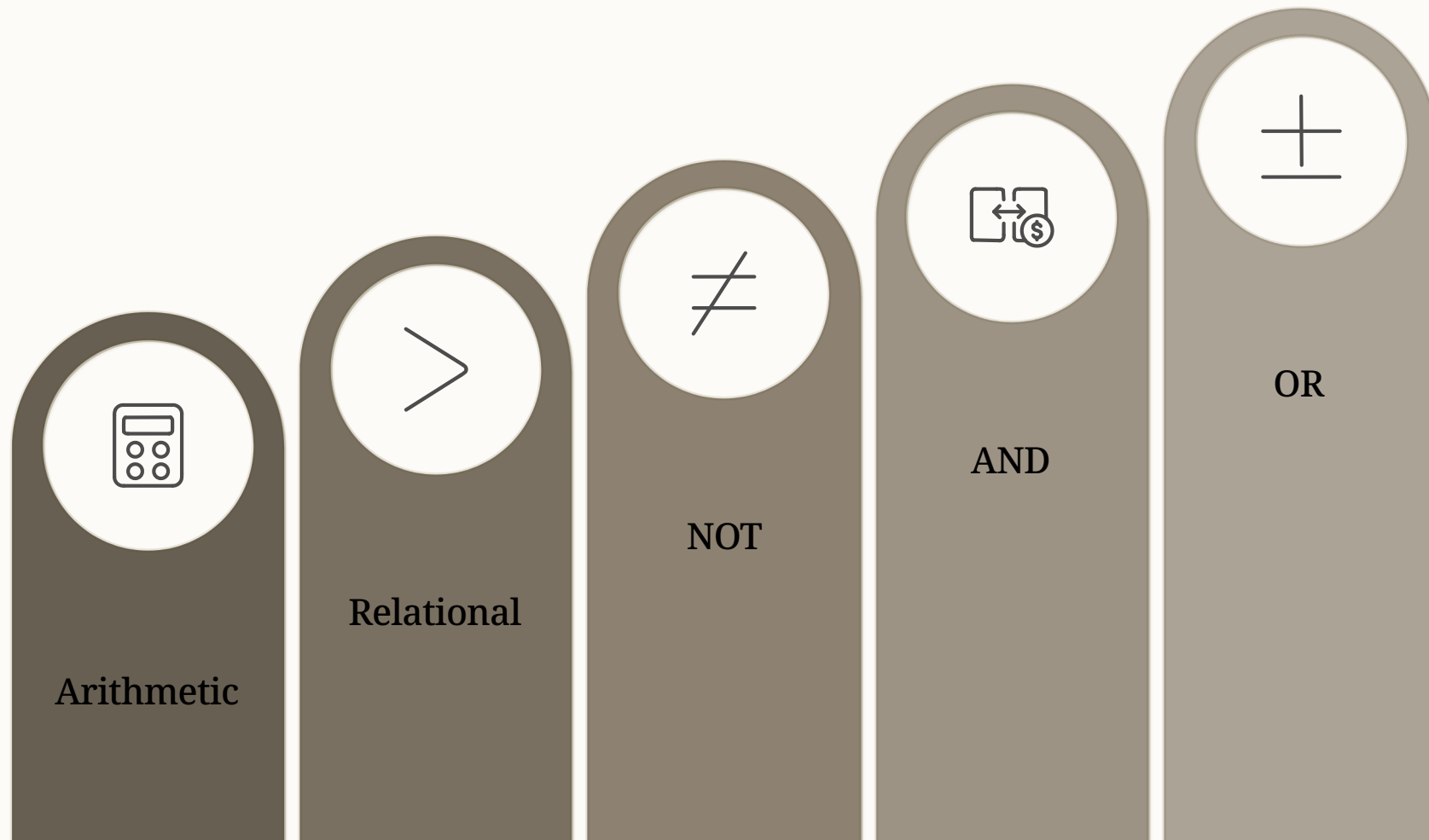
Relational vs Logical Operators

Aspect	Relational Operators	Logical Operators
Purpose	Compare a single pair of values	Combine multiple conditions
Operands	Two values (column vs value)	Two or more conditions
Output	TRUE or FALSE	TRUE or FALSE
Used With	Column names and literals	Relational conditions
Example	<code>marks >= 50</code>	<code>marks >= 50 AND dept='CS'</code>

📌 **Remember:** Relational operators are the building blocks. Logical operators are the connectors that join those blocks into complex conditions.

Operator Precedence in SQL

When multiple operators appear in one query, SQL evaluates them in a fixed order — just like BODMAS in mathematics.



Without Parentheses

WHERE marks > 60 OR dept='HR' AND salary > 30000
AND evaluates first — can give unexpected results.

With Parentheses ✓

WHERE (marks > 60 OR dept='HR') AND salary > 30000

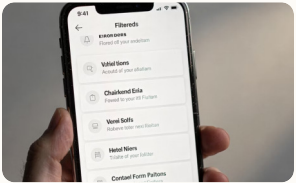
Parentheses override precedence — always use them for clarity.

Real-World Examples

Using a **Student** table (roll, name, marks, dept) and an **Employee** table (emp_id, name, salary, dept, city):

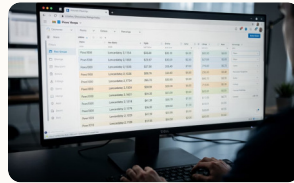
Goal	SQL Query
Students scoring above 70	SELECT * FROM students WHERE marks > 70;
Employees in Delhi or Pune	SELECT * FROM emp WHERE city IN ('Delhi','Pune');
Marks between 50 and 75	SELECT * FROM students WHERE marks BETWEEN 50 AND 75;
Names starting with 'R'	SELECT * FROM students WHERE name LIKE 'R%';
Employees with no city listed	SELECT * FROM emp WHERE city IS NULL;

Where SQL Operators Are Used



SELECT / WHERE

Filter rows using relational and logical operators



HAVING Clause

Filter aggregated groups —
e.g., `HAVING COUNT(*) > 5`



JOIN Conditions

Match rows across tables
using `=` operator



UPDATE / DELETE

Target specific rows: `WHERE emp_id = 101`

Common Mistakes While Using SQL Operators

✗ Using = for NULL

`WHERE email = NULL` always returns nothing. Always use `IS NULL` or `IS NOT NULL`.

✗ BETWEEN with wrong order

`BETWEEN 75 AND 50` returns no rows. Always write the **lower value first**.

✗ LIKE without wildcards

`LIKE 'Raj'` behaves exactly like `= 'Raj'`. Use `%` or `_` for patterns.

✗ AND/OR without parentheses

Relying on precedence leads to logic errors. Always use parentheses to group conditions explicitly.

Practical Scenarios — Choose the Correct Operator

For each scenario below, decide which SQL operator best fits:

1

Marks between 60 and 90

Hint: One operator handles ranges inclusively.

2

Students from Mumbai, Pune, or Nagpur

Hint: Avoids writing three separate OR conditions.

3

Name ends with "kumar"

Hint: Uses a wildcard for partial matching.

4

Salary > 40000 AND department is 'IT'

Hint: Two conditions both must be true.

5

Phone number is missing

Hint: Checks absence of any stored value.

Answers: BETWEEN · IN · LIKE · AND · IS NULL

Classroom Activity

Match the Requirement with the Correct Operator

Requirement	Your Answer	Correct Operator
Find employees with salary from 20K to 50K	_____	BETWEEN
Find students NOT from Delhi	_____	NOT / <>
Products where stock count is missing	_____	IS NULL
Names starting with 'S' and ending with 'a'	_____	LIKE 'S%a'
Employees in HR or Finance departments	_____	IN

 Try writing the full SQL query for at least two of these in your notebook before checking the answer.

Summary Table of All SQL Operators

Category	Operators	Used In	Returns
Arithmetic	+, -, *, /, %	SELECT clause	Numeric value
Relational	=, >, <, >=, <=, <>	WHERE, HAVING	TRUE / FALSE
Logical	AND, OR, NOT	WHERE, HAVING	TRUE / FALSE
Special	IN, BETWEEN, LIKE	WHERE clause	TRUE / FALSE
Special	IS NULL, EXISTS	WHERE clause	TRUE / FALSE

Operators are evaluated in order: **Arithmetic** → **Relational** → **NOT** → **AND** → **OR**. Use parentheses to override.

Quiz + Assignment

Quiz — 5 MCQs

1. Which operator checks a range of values inclusively?
a) IN b) BETWEEN c) LIKE d) EXISTS
2. What does % represent in LIKE?
a) One char b) Zero chars c) Any sequence d) NULL
3. Which is correct to check for NULL?
a) = NULL b) == NULL c) IS NULL d) NULL =
4. Which operator has highest precedence?
a) OR b) AND c) NOT d) *
5. WHERE marks > 50 AND marks < 80 can also be written as?
a) marks IN(50,80) b) marks BETWEEN 50 AND 80 c) marks LIKE 50 d) marks = 50

Assignment

Using the **Student** table (roll, name, marks, city, dept), write SQL queries for:

1. Students with marks between 55 and 85
2. Students from Mumbai, Pune, or Chennai
3. Names starting with the letter 'M'
4. Students in 'CS' department with marks above 70
5. Students whose city is not recorded

☐ Submit in next class. Handwritten or printed both accepted.



Key Takeaways

- **Operators are essential**
They transform basic SELECT queries into precise, powerful data retrieval tools
- **Four categories, one goal**
Arithmetic, Relational, Logical, and Special operators each solve a different problem
- **Precedence matters**
Always use parentheses to make your logic explicit and avoid unexpected query results
- **Avoid NULL traps**
Always use `IS NULL` — never `= NULL`. This is one of the most common beginner errors

→ NEXT LECTURE: SQL EXPRESSIONS

In Lecture 6, we explore **SQL Expressions** — how operators combine with functions and columns to build computed results.